

NumPy - Complete Learning Guide

From Basics to Machine Learning with Book Definitions

Comprehensive NumPy Reference

June 3, 2026

Contents

1	Introduction to NumPy	2
2	NumPy Arrays - The ndarray	3
2.1	Creating Arrays	3
2.2	Array Properties	4
3	Array Operations	5
3.1	Indexing and Slicing	6
4	Mathematical Operations	7
5	Data Manipulation for ML	9
5.1	Reshaping and Transposing	9
5.2	Combining Arrays	9
6	Random Number Generation	11
7	Advanced NumPy for ML	12
7.1	Conditional Logic with np.where	12
7.2	Unique Values and Counting	12
7.3	Saving and Loading Data	12
8	NumPy Cheat Sheet - Complete Reference	14
8.1	Array Creation	14
8.2	Array Properties	14
8.3	Mathematical Operations	15
8.4	Indexing and Selection	15
8.5	Array Manipulation	15
8.6	ML Operations Quick Reference	15
8.7	Memory Tips for ML	16

Chapter 1

Introduction to NumPy

What is NumPy?

Source: "Python Data Science Handbook" by Jake VanderPlas (O'Reilly Media)
"NumPy (Numerical Python) is the fundamental package for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays. NumPy arrays are the foundation upon which the entire Python data science ecosystem is built."

Source: "Python for Data Analysis" by Wes McKinney (O'Reilly Media)
"NumPy is the foundational library for numerical computing in Python. It provides the ndarray object, which is a fast and space-efficient multidimensional array providing vectorized arithmetic operations and sophisticated broadcasting capabilities."

Think of NumPy Like This...

Python Lists = Basic calculator (does the job, but slow for big numbers)

NumPy Arrays = Scientific super-calculator (blazing fast, handles millions of numbers at once!)

Real Analogy: If Python lists are like shopping carts where you can put anything (fruits, clothes, books), NumPy arrays are like specialized storage containers designed for ONE type of item, stacked perfectly for maximum efficiency!

```
1 # Installation
2 # pip install numpy
3
4 import numpy as np
5 print(np.__version__) # Check version
```

Listing 1.1: Getting Started with NumPy

ML Important: Why NumPy for Machine Learning?

NumPy is the backbone of ML libraries like TensorFlow, PyTorch, and Scikit-learn. Your data in ML is ALWAYS represented as NumPy arrays!

Chapter 2

NumPy Arrays - The ndarray

The ndarray Object

Source: "Python Data Science Handbook" by Jake VanderPlas (O'Reilly)
"The NumPy ndarray is a multidimensional array object. It is a table of elements (usually numbers), all of the same type, indexed by a tuple of non-negative integers. In NumPy, dimensions are called axes. The number of axes is the rank of the array. NumPy arrays are stored in contiguous blocks of memory, making them extremely efficient for numerical operations."

2.1 Creating Arrays

```
1 import numpy as np
2
3 # 1D Array
4 arr1d = np.array([1, 2, 3, 4, 5])
5 print(arr1d) # [1 2 3 4 5]
6
7 # 2D Array (matrix)
8 arr2d = np.array([[1, 2, 3], [4, 5, 6]])
9 print(arr2d)
10 # [[1 2 3]
11 #   [4 5 6]]
12
13 # 3D Array (tensor)
14 arr3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
15 print(arr3d.shape) # (2, 2, 2)
```

Listing 2.1: Basic Array Creation

Array Creation Functions

Source: "NumPy Beginner's Guide" by Ivan Idris (Packt Publishing)
"NumPy provides several functions to create arrays with placeholder content. These functions are essential for initializing arrays before computation and are more memory-efficient than creating arrays from Python lists."

```
1 # Zeros and Ones
```

```

2 zeros = np.zeros((3, 4))      # 3 4  array of zeros
3 ones = np.ones((2, 3))       # 2 3  array of ones
4
5 # Identity matrix
6 eye = np.eye(4)               # 4 4  identity matrix
7
8 # Range and linearly spaced
9 range_arr = np.arange(0, 10, 2) # [0 2 4 6 8]
10 linspace = np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1.]
11
12 # Random arrays
13 random_arr = np.random.rand(3, 3) # Uniform [0,1)
14 random_int = np.random.randint(0, 10, (3, 3)) # Random integers
15 normal_arr = np.random.randn(3, 3) # Normal distribution

```

Listing 2.2: Quick Array Creation

ML Important: ML Application: Weight Initialization

When training neural networks, you initialize weights with random values:

```

1 # Neural network weight initialization
2 weights = np.random.randn(784, 128) * 0.01
3 bias = np.zeros((1, 128))

```

2.2 Array Properties

```

1 arr = np.array([[1, 2, 3], [4, 5, 6]])
2
3 print(arr.shape)      # (2, 3) - rows, columns
4 print(arr.ndim)       # 2 - number of dimensions
5 print(arr.size)       # 6 - total elements
6 print(arr.dtype)      # int64 - data type

```

Listing 2.3: Understanding Your Array

Chapter 3

Array Operations

Vectorization

Source: "Python for Data Analysis" by Wes McKinney (O'Reilly)
"Vectorization describes the absence of any explicit looping or indexing in the code. Vectorized operations in NumPy are implemented in C, making them significantly faster than equivalent Python for loops. The key to writing efficient NumPy code is to push loops into compiled NumPy operations."

```
1 a = np.array([1, 2, 3, 4, 5])
2 b = np.array([10, 20, 30, 40, 50])
3
4 # No loops needed!
5 print(a + b)      # [11 22 33 44 55]
6 print(a * b)      # [10 40 90 160 250]
7 print(a ** 2)     # [1 4 9 16 25]
8 print(np.sqrt(a)) # Square root of each element
```

Listing 3.1: Vectorized Operations

Why Vectorization Matters

Python loops are slow. NumPy operations run at C speed because they're written in optimized C code under the hood. One line of NumPy can replace dozens of Python loop lines!

Broadcasting

Source: "Python Data Science Handbook" by Jake VanderPlas (O'Reilly)
"Broadcasting is a set of rules by which NumPy performs arithmetic operations on arrays of different shapes. The smaller array is 'broadcast' across the larger array so that they have compatible shapes. Broadcasting provides a means of vectorizing array operations without making needless copies of data."

```
1 # Scalar broadcasting (easiest to understand!)
2 arr = np.array([1, 2, 3, 4, 5])
3 print(arr + 10)  # [11 12 13 14 15]
4
5 # 1D to 2D broadcasting
```

```

6 matrix = np.array([[1, 2, 3], [4, 5, 6]])
7 row_vec = np.array([10, 20, 30])
8 print(matrix + row_vec)
9 # [[11 22 33]
10 #  [14 25 36]]

```

Listing 3.2: Broadcasting Examples

3.1 Indexing and Slicing

Array Indexing

Source: "NumPy Cookbook" by Ivan Idris (Packt Publishing)

"NumPy offers several ways to index into arrays. Standard Python `x[obj]` syntax can be used, but NumPy extends this to allow indexing with arrays of indices, boolean arrays, and slicing across multiple dimensions simultaneously."

```

1 arr = np.array([[1, 2, 3, 4],
2                 [5, 6, 7, 8],
3                 [9, 10, 11, 12]])
4
5 # Basic indexing
6 print(arr[0, 0])      # 1
7 print(arr[1, 2])      # 7
8
9 # Slicing
10 print(arr[:2])        # First two rows
11 print(arr[:, 1:3])    # All rows, columns 1-2
12
13 # Boolean indexing (POWERFUL!)
14 mask = arr > 5
15 print(arr[mask])      # [6 7 8 9 10 11 12]
16
17 # Fancy indexing
18 indices = [0, 2]
19 print(arr[indices])   # Rows 0 and 2

```

Listing 3.3: Indexing and Selection

ML Important: ML Application: Data Filtering

```

1 # Filter dataset - get only positive class samples
2 data = np.random.randn(100, 5) # 100 samples, 5 features
3 target = np.random.randint(0, 2, 100) # Binary labels
4 positive_samples = data[target == 1] # Boolean indexing!
5 print(f"Positive samples: {positive_samples.shape[0]}")

```

Chapter 4

Mathematical Operations

Statistical Functions

Source: "Python for Data Analysis" by Wes McKinney (O'Reilly)
"NumPy provides fast vectorized implementations of common mathematical and statistical functions. These include basic statistics (mean, median, variance), correlation, and more advanced operations. Most aggregate functions can be computed along a specific axis of a multidimensional array."

```
1 data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
2
3 print(f"Mean: {np.mean(data)}")          # 5.5
4 print(f"Median: {np.median(data)}")      # 5.5
5 print(f"Std: {np.std(data)}")            # 2.87
6 print(f"Min: {np.min(data)}")            # 1
7 print(f"Max: {np.max(data)}")            # 10
8 print(f"Sum: {np.sum(data)}")            # 55
9
10 # Axis operations
11 matrix = np.array([[1, 2, 3], [4, 5, 6]])
12 print(f"Column sums: {np.sum(matrix, axis=0)}") # [5 7 9]
13 print(f"Row means: {np.mean(matrix, axis=1)}")  # [2. 5.]
```

Listing 4.1: Statistical Operations

Linear Algebra in NumPy

Source: "Numerical Python" by Robert Johansson (Apress)
"The numpy.linalg module provides a comprehensive set of linear algebra functions. These functions are built on top of optimized LAPACK and BLAS libraries, providing high-performance implementations of matrix operations, decompositions, and eigenvalue computations essential for scientific computing."

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 # Matrix multiplication (MOST important for ML!)
5 result = A @ B # or np.dot(A, B)
6
7 # Determinant
```



```

8 det = np.linalg.det(A) # -2.0
9
10 # Inverse
11 inv = np.linalg.inv(A)
12
13 # Eigenvalues and eigenvectors
14 eigenvalues, eigenvectors = np.linalg.eig(A)
15
16 # Solve linear equations: Ax = b
17 b = np.array([5, 11])
18 x = np.linalg.solve(A, b)

```

Listing 4.2: Linear Algebra Operations

ML Important: ML Application: Linear Regression in One Line!

```

1 # Linear regression using Normal Equation
2 X = np.array([[1, 1], [1, 2], [1, 3]]) # Add bias column
3 y = np.array([2, 4, 6])
4
5 # One line solution!
6 theta = np.linalg.inv(X.T @ X) @ X.T @ y
7 print(f"Parameters: {theta}") # y = 0 + 2x

```

Chapter 5

Data Manipulation for ML

Array Manipulation

Source: "Python Data Science Handbook" by Jake VanderPlas (O'Reilly)
"NumPy provides many routines for manipulating array shapes: reshaping, flattening, concatenating, and splitting arrays. These operations are essential for preparing data in the correct format for machine learning algorithms, which often expect data in specific shapes and orientations."

5.1 Reshaping and Transposing

```
1 arr = np.arange(12)  # [0 1 2 ... 11]
2
3 # Reshape (total elements MUST match)
4 matrix = arr.reshape(3, 4)  # 3 4
5 tensor = arr.reshape(2, 2, 3)  # 2 23
6
7 # Transpose (swap rows and columns)
8 transposed = matrix.T
9
10 # Flatten (convert to 1D)
11 flat = matrix.flatten()
12
13 # The -1 trick (let NumPy figure out the dimension)
14 col_vector = arr.reshape(-1, 1)  # Column vector
15 row_vector = arr.reshape(1, -1)  # Row vector
```

Listing 5.1: Shape Manipulation

5.2 Combining Arrays

```
1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6], [7, 8]])
3
4 # Vertical stack (add more rows/samples)
5 vstacked = np.vstack((a, b))
6
7 # Horizontal stack (add more columns/features)
```

```
8 hstacked = np.hstack((a, b))
```

Listing 5.2: Stacking and Concatenating

ML Important: ML Application: Adding Bias Term

```
1 X = np.array([[1, 2], [3, 4], [5, 6]]) # Features
2 bias = np.ones((3, 1)) # Bias column
3 X_with_bias = np.hstack((bias, X)) # Add to features
```

Chapter 6

Random Number Generation

Random Number Generation

Source: "NumPy Beginner's Guide" by Ivan Idris (Packt Publishing)
"NumPy's random module provides a comprehensive set of functions for generating random numbers from various probability distributions. Setting a random seed ensures reproducibility of results, which is essential for debugging and scientific research."

```
1 # Set seed for reproducibility (IMPORTANT for ML!)
2 np.random.seed(42)
3
4 # Different distributions
5 uniform = np.random.rand(3, 3)      # Uniform [0,1)
6 normal = np.random.randn(3, 3)      # Standard normal
7 integers = np.random.randint(0, 10, (3, 3)) # Random integers
8
9 # Shuffle data
10 data = np.array([1, 2, 3, 4, 5])
11 np.random.shuffle(data) # In-place shuffle
12
13 # Permutation
14 perm = np.random.permutation(10) # Random ordering of 0-9
```

Listing 6.1: Random Number Generation

ML Important: ML Application: Train-Test Split

```
1 X = np.random.randn(100, 5) # 100 samples, 5 features
2 y = np.random.randint(0, 2, 100) # Labels
3
4 # Shuffle indices
5 indices = np.random.permutation(100)
6 split = int(0.8 * 100) # 80% train
7
8 X_train, X_test = X[indices[:split]], X[indices[split:]]
9 y_train, y_test = y[indices[:split]], y[indices[split:]]
```

Chapter 7

Advanced NumPy for ML

7.1 Conditional Logic with np.where

```
1 arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
2
3 # Find indices
4 indices = np.where(arr > 5)
5 print(indices) # Positions: 5,6,7,8,9
6
7 # IF-ELSE for arrays
8 result = np.where(arr > 5, 'High', 'Low')
9
10 # Replace values
11 filtered = np.where(arr > 5, arr, 0) # Keep if >5, else 0
```

Listing 7.1: np.where - The SQL of NumPy

7.2 Unique Values and Counting

```
1 arr = np.array([1, 2, 2, 3, 3, 3, 4, 5, 5])
2
3 # Unique values
4 unique = np.unique(arr) # [1 2 3 4 5]
5
6 # Count unique values
7 unique, counts = np.unique(arr, return_counts=True)
8 # unique=[1 2 3 4 5], counts=[1 2 3 1 2]
```

Listing 7.2: Finding Unique Elements

7.3 Saving and Loading Data

```
1 # Save array to file
2 arr = np.array([1, 2, 3, 4, 5])
3 np.save('my_array.npy', arr)
4
5 # Load array from file
6 loaded = np.load('my_array.npy')
7
```

```
8 # Save/Load multiple arrays
9 np.savez('arrays.npz', a=arr1, b=arr2)
10 data = np.load('arrays.npz')
11 print(data['a'], data['b'])
12
13 # Save as text (CSV format)
14 np.savetxt('data.csv', arr, delimiter=',')
15 loaded_csv = np.loadtxt('data.csv', delimiter=',')
```

Listing 7.3: File I/O

Chapter 8

NumPy Cheat Sheet - Complete Reference

8.1 Array Creation

Function	Description	Example
<code>np.array(list)</code>	Create from list	<code>np.array([1,2,3])</code>
<code>np.zeros(shape)</code>	Array of zeros	<code>np.zeros((3,4))</code>
<code>np.ones(shape)</code>	Array of ones	<code>np.ones((2,3))</code>
<code>np.eye(n)</code>	Identity matrix	<code>np.eye(3)</code>
<code>np.arange(start,stop,step)</code>	Range with step	<code>np.arange(0,10,2)</code>
<code>np.linspace(start,stop,n)</code>	n evenly spaced	<code>np.linspace(0,1,5)</code>
<code>np.random.rand(d0,d1)</code>	Uniform [0,1)	<code>np.random.rand(3,3)</code>
<code>np.random.randn(d0,d1)</code>	Standard normal	<code>np.random.randn(3,3)</code>

8.2 Array Properties

Property/Method	Description
<code>arr.shape</code>	Tuple of dimensions
<code>arr.ndim</code>	Number of dimensions
<code>arr.size</code>	Total number of elements
<code>arr.dtype</code>	Data type of elements
<code>arr.T</code>	Transpose
<code>arr.reshape(shape)</code>	Change shape
<code>arr.flatten()</code>	Convert to 1D

Function	Description
<code>np.mean(arr)</code>	Mean
<code>np.std(arr)</code>	Standard deviation
<code>np.min(arr) / np.max(arr)</code>	Min/Max
<code>np.sum(arr)</code>	Sum
<code>np.dot(a, b) / a @ b</code>	Matrix multiplication
<code>np.linalg.inv(a)</code>	Matrix inverse
<code>np.linalg.det(a)</code>	Matrix determinant
<code>np.linalg.eig(a)</code>	Eigenvalues/vectors
<code>np.linalg.solve(A, b)</code>	Solve linear equations

Syntax	Description
<code>arr[i, j]</code>	Element at row i, column j
<code>arr[:, j]</code>	All rows, column j
<code>arr[i, :]</code>	Row i, all columns
<code>arr[condition]</code>	Boolean indexing
<code>np.where(condition)</code>	Find indices
<code>arr[[i1, i2]]</code>	Fancy indexing

8.3 Mathematical Operations

8.4 Indexing and Selection

8.5 Array Manipulation

Function	Description
<code>np.vstack((a,b))</code>	Vertical stack
<code>np.hstack((a,b))</code>	Horizontal stack
<code>np.concatenate((a,b), axis=)</code>	Join along axis
<code>np.split(arr, n)</code>	Split into n arrays

8.6 ML Operations Quick Reference

```

1 # 1. Normalize data (Standard Scaling)
2 normalized = (data - np.mean(data, axis=0)) / np.std(data, axis=0)
3
4 # 2. Train-test split
5 indices = np.random.permutation(len(data))
6 split = int(0.8 * len(data))
7 train, test = data[indices[:split]], data[indices[split:]]
8
9 # 3. Add bias term to features
10 X_with_bias = np.hstack((np.ones((n, 1)), X))
11
12 # 4. Linear regression (Normal Equation)
13 theta = np.linalg.inv(X.T @ X) @ X.T @ y
14

```



```

15 # 5. Predictions
16 predictions = X @ theta
17
18 # 6. Mean Squared Error
19 mse = np.mean((predictions - y) ** 2)
20
21 # 7. One-hot encoding
22 one_hot = np.eye(num_classes)[labels]
23
24 # 8. Batch processing
25 for i in range(0, len(data), batch_size):
26     batch = data[i:i+batch_size]

```

Listing 8.1: Common ML Patterns

8.7 Memory Tips for ML

- **Data Matrix X:** Shape $(n_samples, n_features)$
- **Labels y:** Shape $(n_samples,)$
- **Weights w:** Shape $(n_features,)$ or $(n_features, 1)$
- **Predictions:** $X @ w$ gives shape $(n_samples,)$
- **Matrix multiplication:** $(m \times n) @ (n \times p) = (m \times p)$